Day 7 (Evening Session)

Express.js: Server Creation, Routing, Responses and URL Parameters

Lecture notes - Instructor: Abhishek Purohit - Date: 27 Oct 2026 - Notes for: Sumeet Sharma

#	Topic covered in this lecture
1	Recap and the limits of the http module: base URL, sub URLs, one callback for everything
2	Why a library or framework: Express.js (and library vs framework)
3	Setting up an Express project: npm init -y, npm i express, type module
4	Creating and starting an Express server (import express, app, listen)
5	Routing: app.get, app.post, app.put, app.delete; the routes /, /login and /sum
6	Testing with the browser, Postman / Thunder Client; the default request method
7	Parts of a response: status, body, header (metadata); sending JSON; status()
8	Sending data from client to server: URL parameters (/sum/:a/:b) and request.params
9	Factorial route with validation and the 400 status code
10	Questions answered in class (custom status codes, dev dependencies, I/O vs CPU intensive)
11	Revision: summary, definitions, differences, mistakes, interview Q&A, exercises, checklist

How to read these notes
Blue boxes hold the instructor's explanation in clear written English. Yellow boxes are additional explanations added by me; they were **not** said in the lecture. Red boxes list common mistakes. All programs from your class notes are included with explanation and expected output. This lecture did not cover the DOM, events or forms, so those are not part of these notes. Class chatter and screen-sharing breaks are left out.

1. Recap and the Limits of the http Module

So far, with Node.js you can create your own server using the **http module**: the `createServer()` function takes a callback that runs whenever a request arrives, and then you start the server with `listen()`. The instructor first asked whether everyone had created the server using import/export (yes).

```
// recap: server with the core http module
import http from "http";

const server = http.createServer((request, response) => {
    response.write("Hello World");
    response.end();
});

server.listen(6000);       // base URL: 127.0.0.1:6000
```

1.1 Base URL and sub URLs

Instructor's explanation
With this server the only address you can send a request to is **127.0.0.1:6000**, the combination of IP address and port. In a real application (e-commerce, social media, chat, booking) there are many URLs. The domain address such as *myapp.com*, *flipkart.com* or *facebook.com* is called the **base URL**. Whatever comes after the slash is a **sub URL**: *myapp.com/login*, */signup*, */search*, */register*, */profile*, */messages*, */timeline*. A single application can contain fifty, a hundred or two hundred sub URLs, and each URL needs different request processing and a different response.

Example shown in class: on MakeMyTrip the base URL gives the home page, the */hotels* sub URL gives a hotels page, and the railways sub URL gives a different page.

1.2 Why the http module is not enough

Limitation	Explanation from the lecture
One function for everything	createServer has a single callback that accepts every request and generates the response. A request on the base URL, /login, /signup or /profile all reach the same function, so all of them would get the same response.
Conditions for each URL	To give different responses you would have to write conditions on `request.url` inside that single callback: if the URL is the base URL do this, else if it is /login do that. With 100 URLs you would need 100 conditions in one function, which is not an efficient approach.
Conditions for each method	The same URL can behave differently for GET, PUT and DELETE, so you would also have to add conditions on the request method. This makes the job much more difficult as the number of URLs or features grows.
Everything from scratch	Writing everything yourself also means you must handle scalability and security yourself.

Instructor's explanation
For enterprise development we look for something that makes development easier: out-of-the-box functions that do the job in a predefined way, so you simply call a function. It is better to go with a **library or framework** than to build from scratch. You get predefined support for sending different responses, a structure that helps you scale, code that is easier to read and debug, and built-in features that help with security. Building from scratch takes a lot of time and effort because you must think about everything from zero.

2. Express.js

Instructor's explanation
Express is a (small) **framework that simplifies back-end or server development**. It gives you predefined functions, classes and properties. It is available to developers as a module installed with npm (`npm install express`). Using Express you can simplify handling the base URL and many sub URLs, and also other concerns of a web application's back end, which will be explored one by one. For the front end, React JS will be used; for the back end, Express.

Other back-end frameworks in the JavaScript world: **Nest.js**. It depends on you which framework or library you continue with.

2.1 Library vs framework (student question)

Instructor's explanation
A library is a **small bundle of reusable code**: it just contains predefined functions you can use, that is all. A framework is itself a **large ecosystem**: you get end-to-end support to develop the application, including a file and folder structure and built-in support, so you do not need to go to another library for each job. Example: **React is a library** and **Angular is a framework**. Angular gives you everything the front end needs (including routing and sending requests) in itself, whereas with React you go to separate third-party packages for routing and for sending requests. The library is just a small thing; the framework is an entire ecosystem.

Point	Library	Framework
Size	Small collection of reusable code	A large ecosystem
What you get	Only predefined functions	Structure (files/folders), built-in support for most needs
Other packages needed?	Often yes (for example routing in React)	Mostly not (for example Angular)
Example from class	React	Angular

Additional note (not from the lecture)
The lecture calls Express a framework; it is a minimal one that gives you only the web-server basics. Other pieces (database, authentication and so on) come from further packages. The instructor said Express will be explored piece by piece.

3. Setting Up an Express Project

Always use a separate folder for each project. In class the folder was named **my-server**, opened in VS Code with the terminal in that folder. Before writing any code, create package.json.

1. Run `npm init` and answer the prompts, **or** run `npm init -y`. The **-y** flag means yes: accept all the suggested default values (package name from the folder name, version, entry point and the rest) without asking you anything.
2. Open package.json. The description and author are blank (you may fill them in). The main file is index.js. The **type** is commonjs by default; change it to **module** so that the import statement works.
3. Install the dependency locally: `npm i express` (or `npm install express`). This is a dependency of the server application, so it is a local install.
4. npm creates node_modules and package-lock.json, and adds express under **dependencies** in package.json.
5. Create **index.js**, which will hold all the code.

```
npm init -y
npm i express

// package.json (relevant part; the express version on your machine will differ)
{
  "name": "my-server",
  "version": "1.0.0",
  "main": "index.js",
  "type": "module",
  "dependencies": {
    "express": "^x.y.z"
  }
}
```

4. Creating and Starting an Express Server

1. **Import express:** `import express from "express";` No curly braces are used here: we import the entire module and give it a name (the name can be anything; the module name is used for clarity).
2. **Create the application:** `const app = express();` The imported **express** is a function. Calling it creates an **express application, which is nothing but your server**, and returns a reference to it, stored in *app*.
3. **Start the server:** `app.listen(6500);` Pass the port number, as with the http module. Without listen, the server is not started.

Instructor's explanation
If you use the require convention instead, you call `require("express")`, which returns the object reference of the express module, and then you call that reference as a function. The import statement works the same way: import gives the name express and express() creates the application. We no longer need the http module or its createServer function: `express()` does that job.

```
import express from "express";
// const express = require("express");      // CommonJS equivalent

const app = express();     // creates an express application (your server)

app.listen(6500);          // starts the server on port 6500
```

As it stands this server accepts requests but has no code that says what to do for a given URL; that is the next step, routing.

5. Routing

Instructor's explanation
Somewhere we must write logic that says: for which URL, what processing should be done, and what response should be generated. If a request comes on the base URL, process it this way; on /login, process it another way; on /signup another way. This mechanism is called **routing**: a **mapping of your URL (and request method) with the desired processing and response**. Every request has a URL and always one **request method** (GET, POST, PUT, DELETE and so on). With routing you specify, for example, how a GET request on the base URL is processed and how a POST request on /login is processed.

Express gives predefined functions for routing, one for each request method, called on the app reference:

Request method	Express function	Runs when
GET	`app.get(url, callback)`	A GET request arrives on that URL
POST	`app.post(url, callback)`	A POST request arrives on that URL
PUT	`app.put(url, callback)`	A PUT request arrives on that URL
DELETE	`app.delete(url, callback)`	A DELETE request arrives on that URL

Instructor's explanation
Each function takes **two arguments**: the **URL** and a **function**. Read the statement like this: whenever the application receives a (GET) request on this URL, execute this function. That function is the one that accepts the request and generates the response; its first argument is the request object and the second is the response object. You can handle URL one in one way and URL two differently by repeating app.get with a different URL and a different function.

You do **not** write the complete URL (IP and port) in the routes: the IP address and port are already known because of listen. You write only what comes **after the slash**. For the base URL, write just the slash **"/"**.

```
app.get(url1, (request, response) => {
    // process the request for url1 and send a response
});

app.get(url2, (request, response) => {
    // process the request for url2 differently
});
```

The send function: with the http module, sending a response took two calls (write and end). In Express, `response.send(...)` does it in one call; you pass the response to it.

5.1 The first Express application

Purpose: handle the base URL, /login and /sum with different responses.

```
import express from "express";
// const express = require("express");

const app = express(); // it will create an express application
// and this express application is nothing but your server

//127.0.0.1:6500/
app.get("/", (request, response) => {
    response.send("Hello World");
});

app.get("/login", (request, response) => {
    response.send("welcome to login");
});

app.get("/sum", (request, response) => {
    var x = 4;
```

```
    var y = 7;
    var z = x + y;
    response.send(`Sum is ${z}`);
});

app.listen(6500);
```

Explanation: three GET routes are registered. The route "/" answers the base URL with Hello World. The route "/login" answers with a different string. The route "/sum" adds two numbers fixed by the developer (4 and 7) and sends the text using a template literal (backticks). Finally listen starts the server.

Run: `node index.js` (index.js is the starting point). No error appears and the server keeps listening on port 6500.

```
Output:
127.0.0.1:6500/         ->  Hello World
127.0.0.1:6500/login    ->  welcome to login
127.0.0.1:6500/sum      ->  Sum is 11
```

Common mistake
After changing the source code you must **restart the server**: stop it with Ctrl + C and start it again with node index.js. Otherwise the old code keeps running (the instructor did this after adding the /sum route).

5.2 Which request method is used by default?

Instructor's explanation
We never specified a request method when typing URLs in the browser. The browser sends a **GET request by default**. That is why app.get handled our requests; it is the proof that the server is handling the GET request the browser sent. To handle POST, PUT or DELETE you write app.post, app.put or app.delete (the same structure works in the same way).

Always write the slash for the base URL. A student asked whether the "/" is compulsory; the instructor said it is not a good practice to leave a route empty. At deployment, code quality tools such as SonarQube report broken or empty links. Write the base URL route as "/".

6. Testing Routes: Browser vs API Clients

A browser can only send GET requests easily and shows you just the response body. To test every request method and to see every technical detail, use an **API client** such as **Postman**, **Thunder Client** (a VS Code extension) or Boomerang. In class a Postman extension in VS Code was used (it needed a login). The old Postman Chrome app is no longer supported.

- Choose the **request method** from the drop-down (GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS).
- Enter the URL, for example **http://127.0.0.1:6500/** (include http://), and click **Send**.
- At the bottom you see the response: body, status (200), time taken (about 12 ms for the first request, 15 ms for another), size in bytes, and headers.
- If the response looks old after a code change, restart the server and send again.

Additional note (not from the lecture)
The instructor said that in real projects Postman (or a similar client) is used directly. The browser shows only the body; tools like Postman show body, status, headers, time and size. A student asked whether 'request body and response body' are related terms; the instructor pointed out that request and response each have their own parts.

7. Parts of a Response

Part	What it is	Example from class
Status	A numeric code telling whether the request succeeded	200 (OK); default status of a response
Body	The actual data of the response (earlier we called it response data)	Hello World (string) or a JSON object
Header	**Metadata** about the response: data about data	X-Powered-By: Express, Content-Type, ETag, Date, Connection

Instructor's explanation
Metadata means **data about data**. If the response is our data, then its size, who sent it, what type it is, and at what time it was sent are metadata. The server sends all of them along with the response, and tools like Postman let you see them in the headers.

7.1 Content type: text vs JSON

Our first responses were plain strings. For the route "/login" the header **Content-Type** is **text/html**. But in real applications the server should send a **JSON response**, so that the content type is **application/json**. Modern applications use the JSON format.

To send JSON, pass a JSON object to **send**. Example: a key *message* with the value *Hello world*. Express then sets the content type to application/json.

7.2 Setting the status code: status()

Instructor's explanation
By default a response has status **200**. As a back-end developer you can decide the status code with the **status()** function, for example 200, 201, 202, 400, 404 or 500. A response can be a failure or a success, so whenever you send a response you can also decide its status. Call `status()` and then chain `send()` to it.

```
import express from "express";

const app = express();

//127.0.0.1:6500/
app.get("/", (request, response) => {
    response.status(200).send({ message: "Hello world" });
});

app.get("/login", (request, response) => {
    response.send("welcome to login");
});

app.get("/sum", (request, response) => {
    var x = 4;
    var y = 7;
    var z = x + y;
    response.send(`Sum is ${z}`);
});

app.listen(6500);
```

Explanation: the base route now sets status 200 and sends a JSON object. The /login and /sum routes are unchanged and still send strings. Restart the server after editing.

```
Output:
GET /        ->  status 200, Content-Type: application/json
```

```
                body: { "message": "Hello world" }
GET /login   ->  status 200, Content-Type: text/html
                body: welcome to login
```

In class, the JSON response from the base URL showed the header **Content-Type: application/json** and **X-Powered-By: Express**, while /login showed **text/html**.

Status code range	Meaning (commonly used)	Used in class
200 / 201 / 202	Success family	200 default success
400	Bad request (the client sent invalid data)	Negative factorial input
404	Not found	Mentioned
500	Server error	Mentioned

Additional note (not from the lecture)
The meanings in the second column are the standard meanings; the instructor listed the numbers, and explained 400 as bad request.

8. Sending Data from Client to Server: URL Parameters

So far the numbers 4 and 7 were chosen by the developer. In a real use case the **client sends data** to the server, the server receives and processes it, and then generates a response. The question: how does a client send data to the server?

Instructor's explanation
Example: on MakeMyTrip, searching for a flight from Delhi to Leh puts the inputs into the URL: the itinerary (Delhi airport code and Leh airport code), the travel date, trip type, person type, cabin class and language. The server reads these values from the URL and generates the response. So **data can be sent on the URL**; this is one way to send data from a client to the server.

Common mistake
Never send sensitive data on the URL (login passwords, OTPs, payment data). Anything on the URL is exposed. Sending a source, destination or travel date on the URL is harmless. Sensitive data is sent in another way (later topics).

The modern convention for passing data on the URL is called a **URL parameter**. Example: `/sum/5/8`, where **5 and 8 are URL parameters** (it could also be 3 and 4, 10 and 20, 0 and 2). URL parameters can be numeric, string, date or Boolean data.

8.1 Declaring URL parameters in a route

Instructor's explanation
The server must know which part of the URL is a parameter. In the route, write a **colon** followed by a **name** for each parameter: `/sum/:a/:b`. The name can be anything (a and b, n1 and n2, x and y), but you need it to read the value in your logic. A route can have as many parameters as you need.

8.2 Reading the parameters: request.params

Instructor's explanation
Express prepares an **object of all the URL parameters**, with the parameter names as keys and the values from the client's URL as values. For a request `/sum/3/5` the object looks like `{ a: "3", b: "5" }`. This object is available on the **request** object (because the client sends it, the request carries it), in the property **params**: `request.params`.

Phase 1: first attempt (strings are joined, not added)

```
app.get("/sum/:a/:b", (request, response) => {
    const paramsData = request.params;                  // { a: "3", b: "5" }
    const sum = paramsData.a + paramsData.b;            // strings!
    response.send({ sum: sum });
});

Output:
GET /sum/3/5   ->   { "sum": "35" }
```

Common mistake
URL parameter values are read as **strings**. The + operator joins strings, so 3 and 5 give "35". This is the same string-to-number issue seen earlier in JavaScript.

Phase 2: parse the values into integers

```
app.get("/sum/:a/:b", (request, response) => {
    const paramsData = request.params;
    const sum = parseInt(paramsData.a) + parseInt(paramsData.b);   // numeric addition
    response.send({ sum: sum });
});
```

Explanation: the route declares two parameters, a and b. request.params gives the object. parseInt converts each string to a number so that + adds them. The result is sent as a JSON object. Restart the server after the change.

```
Output:
GET /sum/3/5   ->   { "sum": 8 }
GET /sum/7/5   ->   { "sum": 12 }
```

What if the client sends letters? (student question)

You can pass any text, for example `/sum/abc/xyz`: abc goes to the first parameter and xyz to the second. parseInt of an alphabetic value gives **NaN** (not a number), and NaN + NaN is NaN, so the response contains NaN.

Instructor's explanation
To stop invalid input like this, **validate** the parameters, for example with a **regular expression** that accepts only numbers (the instructor answered 'regular expression' to a student's question and said validation is a topic for later).

Additional note (not from the lecture)
As a small illustration (not shown in class), you could test with `isNaN(parseInt(value))` and send a 400 response. Express can also restrict a route parameter with a regular expression in the route path.

9. Factorial Route with Validation

Purpose: take a number on the URL, compute its factorial, and return it as JSON. Reject a negative number with a 400 status.

Basic version

```
app.get("/factorial/:num", (request, response) => {
    const n = parseInt(request.params.num);     // string -> number
    let fact = 1;
    for (let i = 1; i <= n; i++) {
        fact = fact * i;
    }
    response.send({ factorial: fact });
});
```

Explanation: the route has a single parameter named num, so request.params contains one key and you read it directly as **request.params.num**. The loop multiplies fact by every i from 1 to n, and the result is sent as JSON.

```
Output:
GET /factorial/5   ->   { "factorial": 120 }
```

Problem and improvement: negative input

Instructor's explanation
What if the client sends a negative value? Factorial is not defined for it, so we must reject the request by saying it is a **bad request** and send a message that negative values are not allowed. If the value is not negative, do the processing. By default a response has status 200, but this request carries invalid data from the client, so the status code should be **400 (bad request)**. Valid input still returns 200.

```
app.get("/factorial/:num", (request, response) => {
    const n = parseInt(request.params.num);

    if (n < 0) {                                          // invalid input
        response.status(400).send({ message: "Negative values are not allowed" });
    } else {                                              // valid input
        let fact = 1;
```

```
        for (let i = 1; i <= n; i++) {
            fact = fact * i;
        }
        response.send({ factorial: fact });              // status 200 by default
    }
});

Output:
GET /factorial/5    ->  status 200   { "factorial": 120 }
GET /factorial/-2   ->  status 400   { "message": "Negative values are not allowed" }
```

Additional note (not from the lecture)
In class the instructor first mentioned 'n less than or equal to zero' and then spoke of n less than zero; the code above rejects only negative numbers, so factorial of 0 correctly returns 1.

10. Questions Answered in Class

Question	Instructor's answer
Can we use custom status codes?	No, only the predefined ones: they start at the hundreds and go up to the 500 series (1xx to 5xx).
What is the difference between dependency and dev dependency?	Will be covered later. In short, dev dependencies are those you need only during development (for example TypeScript or unit-testing packages) and that should not go into the production environment; marking them as dev dependencies keeps them on the developer machine only.
Is Node.js good for every kind of application?	Node.js is best for **I/O intensive** (input/output) applications. For **CPU intensive** work (heavy data processing, big data analysis) JavaScript suffers because it is single threaded and you would need worker threads, whereas Java gives a multithreaded system. A separate topic; search these two concepts.
Can we test in Postman?	Yes. Postman, Thunder Client or Boomerang; they only send a request and show the response.

11. Revision Material

11.1 Summary of all major topics

- **http module limits:** one callback for all URLs and methods; you would need a condition per URL and per method. A real app has many sub URLs (base URL + sub URL).
- Use a library or framework for enterprise development: predefined functions, scalability, readability, security. **Library** = small reusable code (React). **Framework** = whole ecosystem (Angular).
- **Express** is a framework that simplifies back-end / server development. Install with npm; Nest.js is another option.
- **Setup:** npm init -y, change type to module, npm i express, create index.js.
- **Server:** import express from "express"; const app = express(); app.listen(port);
- **Routing** = mapping URL + request method to processing and response: app.get, app.post, app.put, app.delete (url, callback(request, response)). Use "/" for the base URL.
- response.send() writes and ends the response in one call. Restart the server after editing code. Browsers send GET by default.
- API clients (Postman, Thunder Client) show body, status, headers, time, size and let you choose the method.
- **Response parts:** status (default 200), body (data), header (metadata, such as Content-Type). Send JSON by passing an object; set the status with response.status(code).
- **URL parameters:** data in the URL, declared with :name in the route, read from request.params as strings (use parseInt). No sensitive data on the URL.
- Validate input and reply with 400 for bad requests (negative factorial). Custom status codes are not allowed.

11.2 Important definitions and syntax

Term / Syntax	Meaning
Base URL	Domain (or IP and port) with nothing after the slash
Sub URL	What comes after the slash, for example /login
Express	Framework that simplifies back-end development
Routing	Mapping URL + request method to processing and response
`import express from "express";`	Import the express module
`const app = express();`	Create an express application (the server)
`app.listen(6500);`	Start the server on port 6500
`app.get("/path", (request, response) => {});`	Handle a GET request on /path (similarly post, put, delete)
`response.send(data)`	Send the response (string or JSON object)
`response.status(400).send({...})`	Set a status code and send JSON
`"/sum/:a/:b"`	Route with two URL parameters named a and b
`request.params`	Object of URL parameters (keys = names, values = strings)
`npm init -y`	Create package.json with default values
Header	Metadata of the response (content type, size, date and so on)
Status code	Number telling the result of the request (200, 400, ...)

11.3 Key differences between related concepts

Concept A	Concept B	Key difference
http module server	Express server	One callback and manual if-conditions vs one route function per URL and method
response.write + response.end	response.send	Two calls vs one call
Library	Framework	Small set of functions vs entire ecosystem
Base URL	Sub URL	Nothing after the slash vs a path after the slash
Response body	Response header	The data vs metadata about the data
text/html	application/json	String response vs JSON response
Browser	Postman	Only GET and only body vs any method plus status, headers, time, size
npm init	npm init -y	Asks questions vs accepts all default values
Dependency	Dev dependency	Needed in production vs only needed during development
I/O intensive	CPU intensive	Node.js is best for the first; the second needs multithreading / worker threads

11.4 Common mistakes to avoid

Mistake	Why it fails	Correction
Not restarting the server after changing code	Old code keeps running	Ctrl + C, then node index.js
Using import while package.json says commonjs	Cannot use import statement outside a module	Set "type": "module"
Forgetting app.listen	The server never starts	Call app.listen(port)
Writing the full URL (127.0.0.1:6500) in the route	Routes take only the path	Use "/login" and "/"
Leaving the base URL route empty	Quality tools flag it; bad practice	Use "/"
Adding URL parameters without parseInt	Values are strings: 3 and 5 give "35"	parseInt (or Number)
Sending sensitive data on the URL	It is exposed	Do not put passwords or OTPs on the URL
Sending 200 for invalid input	Misleads the client	Use status(400) for a bad request
Trying a custom status code	Only predefined codes exist	Use 1xx to 5xx codes
Testing POST/PUT/DELETE in the address bar	Browser sends GET	Use Postman or Thunder Client

11.5 Interview questions and answers

Items marked (extra) go beyond the lecture.

Question	Answer
Why is the http module not enough for big applications?	One callback handles every request, so you need many conditions on URL and method; Express provides routing to avoid this.
What is Express?	A framework that simplifies back-end / server development in Node.js.
Library vs framework?	A library is small reusable code (React); a framework is an entire ecosystem with structure and built-in support (Angular).
How do you create an Express server?	import express; const app = express(); app.listen(port);
What is routing?	Mapping a URL and request method to the processing and response.
Which functions implement routing?	app.get, app.post, app.put, app.delete.
Which method does a browser use by default?	GET.
What does response.send do?	Sends the response (string or JSON) in one call, replacing write and end.
What are the parts of a response?	Status, body and header (metadata).
How do you send a JSON response with a status?	response.status(200).send({ message: "Hello world" });
What is the default status code?	200.
What does -y mean in npm init -y?	Accept all default values without prompting.
How do you declare a URL parameter?	With a colon and a name in the route, for example "/sum/:a/:b".
How do you read URL parameters?	From request.params, an object of keys and values; values are strings.
Why parseInt on URL parameters?	They are strings; without parseInt + concatenates them.

Question	Answer
Should sensitive data go on the URL?	No; the URL is exposed.
Which status code for invalid client input?	400 Bad Request.
Can you create custom status codes?	No, only the predefined ones.
(extra) Difference between URL parameters and query strings?	URL (path) parameters are part of the path, like /sum/3/5; query strings come after a question mark (as seen in the MakeMyTrip URL).

11.6 Practice exercises (solutions not included)

Easy

1. Create a project folder, run npm init -y, set type to module and install express.
2. Create an Express server on port 6500 with a base route that sends the text Welcome.
3. Add a /about route that sends a JSON object with your name.

Medium

1. Add a /product/:a/:b route that returns the product of the two numbers as JSON, parsing the values correctly.
2. Send a request to /about in Postman and note the status, body, content type header and response time.
3. Change the /about route to set status 201 and verify it in Postman.
4. What do you get for /sum/abc/xyz? Explain why.

Challenging

1. Create a /power/:base/:exp route that returns base raised to the exponent and rejects a negative exponent with 400 and a message.
2. Add validation so that non-numeric parameters get a 400 response instead of NaN.
3. Create a /greet/:name route with a string parameter and respond with a greeting in JSON.
4. Write the same /factorial route with app.post and think about how you would test it (hint: which tool?).

11.7 Final revision checklist

- [ ] I can explain the limits of the http module (single callback, many URLs, many methods).
- [ ] I can distinguish base URL and sub URL.
- [ ] I can explain library vs framework with React and Angular.
- [ ] I can set up an Express project (npm init -y, type module, npm i express).
- [ ] I can create and start an Express server.
- [ ] I can explain routing and use app.get, app.post, app.put, app.delete.
- [ ] I know the browser sends GET by default and I can test with Postman / Thunder Client.
- [ ] I can explain the status, body and header of a response and send JSON.
- [ ] I can set a status code with response.status().
- [ ] I can declare URL parameters and read them with request.params (with parseInt).
- [ ] I can explain why sensitive data should not go on the URL.
- [ ] I can write the factorial route with 400 validation.

- [ ] I know custom status codes are not allowed, and what I/O intensive vs CPU intensive means.